

Architecture Decision Records

Version: 1.0

Purpose

This directory contains Architecture Decision Records for Trading Platform Pro and its primary application, the Trading Cockpit.

Architecture Decision Records preserve the context and rationale of significant architecture decisions.

An ADR answers:

> Why was this architecture decision made?

Current architecture is documented in:

- `docs/architecture/Architecture.md`
- `docs/architecture/Domain\_Model.md`
- `docs/architecture/Infrastructure.md`
- `docs/architecture/Project\_Structure.md`

ADRs preserve decision history.

They do not replace current architecture documentation.

Product Direction

Architecture decisions shall support the Trading Cockpit as the primary product.

Current architecture follows:

```text

Trading Cockpit Requirement

↓

Vertical Product Capability

↓

Application and Domain Behaviour

↓

Required Infrastructure

```

Shared platform capabilities may evolve when multiple concrete product capabilities require them. Do not create architecture decisions solely for hypothetical future applications or speculative scalability.

What Requires an ADR

Create an ADR when a decision has significant and durable architecture impact.

Examples:

- architecture dependency direction
- major capability boundaries
- Domain ownership changes
- Application workflow ownership changes
- persistence architecture
- broker integration architecture
- market data integration architecture
- runtime lifecycle architecture
- AsyncIO integration strategy
- dependency injection strategy
- major framework selection
- cross-capability infrastructure
- significant external integration strategy

An ADR is particularly useful when:

- multiple materially different solutions exist
- the selected solution introduces durable constraints
- future developers may reasonably question the decision
- reversing the decision would require significant migration

What Does Not Require an ADR

Do not create an ADR for every implementation change.

An ADR is normally not required for:

- bug fixes
- formatting changes
- documentation corrections
- local refactoring
- test additions
- naming changes
- small internal implementation details
- configuration value changes
- routine dependency updates

Do not create ADRs solely to document that code was changed.

Implementation behaviour belongs in the relevant capability documentation.

ADR and Architecture Documentation

Architecture documentation describes the current intended architecture.

ADR documentation describes why a significant architecture decision was made.

Example:

```
```text
```

Architecture.md

→ Current dependency direction

ADR

→ Why this dependency direction was selected

```
```
```

When an accepted ADR changes current architecture:

1. Create or update the ADR.
2. Update affected architecture documentation.
3. Update related technical documentation.
4. Implement the architecture change.
5. Add or update architecture tests where practical.

Do not leave current architecture documentation dependent on reading historical ADRs.

ADR Immutability

Accepted ADRs preserve historical decision context.

After an ADR is accepted, do not rewrite its decision history to make it appear that the original decision was different.

Minor corrections may be made for:

- spelling
- broken references
- formatting
- factual metadata errors

A materially changed architecture decision requires:

- a new ADR
- explicit reference to the previous ADR
- status update of the superseded ADR where appropriate

Architecture history shall remain traceable.

ADR Status

Use one of the following statuses:

```
```text
Proposed
Accepted
Superseded
Deprecated
Rejected
```
```

Proposed

The decision is under consideration.

A Proposed ADR may still change materially.

Implementation shall not treat a Proposed ADR as an accepted architecture contract unless the development workflow explicitly requires an approved prototype.

Accepted

The decision is approved and represents an active architecture decision.

Current architecture documentation shall be consistent with accepted ADRs.

Superseded

The decision has been replaced by another ADR.

A Superseded ADR shall identify the replacing ADR.

Example:

```
```text
Status: Superseded
Superseded by: ADR-0007
```
```

Do not delete superseded ADRs.

Deprecated

The decision is no longer recommended or is being phased out.

Deprecated may be used when migration is incomplete and no single replacement ADR fully supersedes the decision.

The ADR shall explain the current migration context.

Rejected

The proposed decision was evaluated and explicitly not selected.

Rejected ADRs may be preserved when the rationale provides durable architecture context.

Do not create Rejected ADRs for every informal alternative discussed during development.

ADR Numbering

ADR files shall use sequential four-digit numbering.

Format:

```
```text
NNNN-short-decision-title.md
```
```

Examples:

```
```text
0001-use-pyside6.md
0002-use-asyncio-runtime.md
0003-use-sqlalchemy-persistence.md
0004-separate-broker-and-market-data-lifecycles.md
```
```

Numbers shall not be reused.

A removed or rejected ADR number remains part of the architecture history.

File Naming

Use:

- lowercase
- hyphen-separated words
- concise architecture decision description

Prefer:

```
```text
0005-use-vertical-product-slices.md
```
```

Avoid:

```
```text
ADR5.md
decision_new.md
architecture_change_final.md
```
```

The filename shall communicate the primary decision.

ADR Title

The ADR title shall describe the decision.

Example:

```
```markdown
```

### **ADR-0005: Use Vertical Product Slices**

```
```
```

Prefer decision-oriented titles.

Avoid vague titles such as:

```
```text
Architecture Update
Platform Changes
New Design
```
```

Required ADR Structure

Every ADR shall use the following structure:

```
```markdown
```

### **ADR-NNNN: Decision Title**

Status: Proposed

Date: YYYY-MM-DD

---

## Context

Describe the concrete product or technical problem.

Explain:

- current situation
- affected capability
- architecture constraint
- why a durable decision is required

---

## Decision

Describe the selected architecture decision.

State the decision explicitly.

---

## Rationale

Explain why the selected decision was chosen.

Describe the important decision criteria.

---

## Alternatives Considered

### Alternative A

Describe the alternative.

Explain why it was not selected.

### Alternative B

Describe the alternative.

Explain why it was not selected.

---

## Consequences

### Positive Consequences

Describe expected benefits.

### Negative Consequences

Describe accepted costs or limitations.

---

## Implementation Impact

Identify affected architecture areas.

Examples:

- Domain
- Application
- Infrastructure
- Presentation
- Runtime
- Persistence
- Configuration
- Testing

Do not list unaffected areas solely to make the ADR appear comprehensive.

---

## Migration

Describe migration requirements where applicable.

If no migration is required:

No migration is required.

---

## Validation

Describe how architecture compliance can be verified.

Examples:

- architecture tests
- dependency checks
- integration tests
- runtime tests
- documentation review

---

## Related Documents

- Architecture.md
- Infrastructure.md

---

Additional sections may be added when the decision requires them.

Do not remove the core decision context.

---

## Context

The Context section shall describe the problem before describing the selected solution.

Avoid writing the Context section as a justification of a decision that has already been assumed.

Good context identifies:

- concrete requirement
- current limitation
- affected capability
- relevant architecture constraint

Do not use hypothetical future scale as the only context unless a concrete approved requirement exists.

---

## Decision

The Decision section shall be explicit.

Prefer:

```text

The Trading Cockpit shall organize product behaviour around vertical product capabilities while preserving architecture dependency direction.

```

Avoid:

```text

We should probably consider a more modular architecture.

```

The selected architecture direction shall be unambiguous.

---

## Rationale

The Rationale section explains why the decision was selected.

Relevant criteria may include:

- product requirement

- architecture consistency
- capability ownership
- maintainability
- testability
- operational safety
- migration cost
- runtime behaviour

Do not use generic claims such as:

```
```text
```

More scalable

More enterprise-ready

More modern

```
```
```

without a concrete project-specific reason.

---

## Alternatives Considered

Document materially relevant alternatives.

An alternative should have been realistic enough to affect the decision.

Do not invent artificial alternatives solely to complete the ADR template.

For each alternative explain:

- what the alternative is
- relevant advantages
- why it was not selected

---

## Consequences

Every significant architecture decision has consequences.

Document both:

- positive consequences
- negative consequences

Do not describe an architecture decision as having only benefits.

Negative consequences may include:

- additional translation code
- migration effort
- stronger ownership rules
- more explicit interfaces
- reduced generic reuse
- operational complexity

Accepted trade-offs are part of architecture history.

---

## Trading-Critical Decisions

Architecture decisions affecting trading-critical workflows require additional analysis.

Relevant areas include:

- order submission
- order cancellation
- broker acknowledgement
- execution processing
- position lifecycle
- reconciliation
- PAPER and LIVE separation

The ADR shall evaluate where relevant:

- external side effects
- duplicate submission risk
- retry behaviour

- unknown broker state
- disconnect behaviour
- reconnect behaviour
- persistence impact
- reconciliation impact
- PAPER impact
- LIVE impact

Do not approve a trading-critical architecture decision based only on code organization.

---

## Broker Architecture Decisions

Broker-related ADRs shall preserve the distinction between:

```text

Application workflow ownership

↓

Broker port

↓

Infrastructure adapter

↓

Provider API

```

Broker adapters execute technical provider communication.

They do not own:

- trading decisions
- duplicate submission policy
- portfolio risk decisions
- position close decisions

A reconnect architecture shall not silently imply order resubmission.

---

## Market Data Architecture Decisions

Market data ADRs shall evaluate:

- provider communication
- source timestamps
- freshness semantics
- unavailable data
- subscription ownership
- subscription lifecycle

Market data architecture shall remain distinguishable from broker execution architecture.

A shared provider does not automatically mean a shared lifecycle.

---

## Persistence Architecture Decisions

Persistence ADRs shall evaluate:

- Domain identity
- repository contracts
- transaction ownership
- persistence model translation
- migration impact
- reconciliation impact

Persistence models shall remain Infrastructure concerns.

SQLAlchemy models shall not become Domain Entities.

Architecture decisions shall not silently rewrite trading history.

---

## Runtime Architecture Decisions



Runtime ADRs shall evaluate:

- startup order
- shutdown order
- task ownership
- cancellation
- timeout behaviour
- degraded state
- broker lifecycle
- market data lifecycle
- AsyncIO event loop impact
- UI thread impact

Long-running tasks require explicit lifecycle ownership.

Do not approve unmanaged background execution as architecture solely because it is technically simple.

---

## Architecture Decision Process

Before creating an ADR:

1. Identify the concrete requirement.
2. Inspect the current implementation.
3. Review current architecture documentation.
4. Identify the owning capability.
5. Identify affected architecture boundaries.
6. Identify real consumers.
7. Evaluate materially different alternatives.
8. Evaluate migration impact.
9. Evaluate trading safety where relevant.
10. Draft the ADR as `Proposed`.

Before accepting an ADR:

1. Review the decision.
2. Review alternatives.
3. Review consequences.
4. Review implementation impact.
5. Review migration.
6. Review validation.
7. Update status to `Accepted`.
8. Synchronize current architecture documentation.

---

## ADR Review

ADR review shall verify:

- concrete requirement identified
- decision is architecture-significant
- owning capability identified
- current architecture inspected
- real consumers identified
- decision is explicit
- rationale is project-specific
- realistic alternatives evaluated
- positive consequences documented
- negative consequences documented
- migration impact evaluated
- validation defined
- trading safety evaluated where relevant
- current architecture documentation synchronized

Do not accept an ADR solely because the proposed implementation already exists.

---

## ADR Lifecycle

Typical lifecycle:

```text

Proposed

↓

Accepted

```

A later decision may produce:

```text

Accepted ADR

↓

New ADR Accepted

↓

Previous ADR Superseded

```

or:

```text

Accepted ADR

↓

Architecture phased out

↓

Deprecated

```

Rejected proposals may remain:

```text

Proposed

↓

Rejected

```

Status transitions shall remain explicit.

---

## Superseding an ADR

When replacing an accepted ADR:

1. Create a new ADR.
2. Explain the new context.
3. Reference the previous ADR.
4. Accept the new ADR.
5. Change the previous ADR status to `Superseded`.
6. Add `Superseded by`.
7. Update current architecture documentation.
8. Update related technical documentation.
9. Implement migration where required.

Do not rewrite the previous ADR decision to match the new architecture.

---

## ADR References

Use explicit ADR identifiers.

Example:

```text

ADR-0005

```

When one ADR supersedes another, reference both identifier and decision where useful.

Example:

```
```text
Supersedes ADR-0002: Use Shared Provider Lifecycle
```
```

Avoid references such as:

```
```text
the old ADR
previous decision
```
```

Architecture history shall remain traceable.

---

## Documentation Generation

ADR Markdown files are part of the documentation source.

When ADR documentation changes run:

```
```bash
python scripts/generate_docs.py
```
```

The command shall complete successfully.

Generated DOCX and PDF files are derived artifacts.

Do not manually edit generated ADR documentation.

---

## ADR Repository Rules

Before committing an ADR verify:

- filename follows numbering convention
- ADR identifier matches filename
- status is valid
- date is present
- context is concrete
- decision is explicit
- rationale is project-specific
- alternatives are realistic
- consequences include trade-offs
- implementation impact is identified
- migration is addressed
- validation is defined
- related documentation is synchronized
- documentation generation passes

---

## ADR Review Checklist

Before accepting an ADR verify:

- architecture-significant decision exists
- concrete requirement identified
- Trading Cockpit impact identified
- owning capability identified
- current implementation inspected
- current architecture reviewed
- real consumers identified
- speculative future requirements avoided
- decision explicit
- rationale concrete
- alternatives realistic
- positive consequences documented
- negative consequences documented
- architecture boundaries preserved

- external side effects evaluated where relevant
- duplicate submission risk evaluated where relevant
- retry behaviour evaluated where relevant
- broker acknowledgement evaluated where relevant
- reconciliation impact evaluated where relevant
- PAPER and LIVE impact evaluated where relevant
- migration defined
- validation defined
- architecture documentation synchronized
- documentation generation passed

---

## **Related Documents**

- Product\_Vision.md
- Product\_Roadmap.md
- Project\_Overview.md
- Architecture.md
- Domain\_Model.md
- Infrastructure.md
- Project\_Structure.md
- Technical\_Specifications.md
- API\_Guidelines.md
- Coding\_Standards.md
- Development\_Guidelines.md
- Testing\_Strategy.md
- AI\_Agent\_Guide.md
- AGENTS.md